

C++ Programming

Day 13: File I/O와 Exception Handling

ofstream, ifstream, ios::app, File Parsing, try/catch, throw

Contents

1	학습 목표	2
2	Day 13 개요	2
3	File Output: ofstream	2
3.1	파일 열기와 데이터 쓰기	2
3.2	최종 코드	3
3.3	실습	3
4	File Input: ifstream	4
4.1	파일 열기	4
4.2	getline으로 한 줄 읽기	4
4.3	최종 코드	5
4.4	실습	5
5	Append Mode: ios::app	6
5.1	기본 구조	6
5.2	최종 코드	6
5.3	실습	7
6	File Parsing	7
6.1	스트림 추출 연산자로 데이터 분리	7
6.2	읽은 데이터 계산하기	8
6.3	최종 코드	8
6.4	실습	9
7	Exception Handling: try와 catch	10
7.1	기본 구조	10
7.2	최종 코드	10
7.3	실습	11
8	throw와 Standard Exception	11
8.1	runtime_error	11
8.2	함수에서 예외 발생시키기	12
8.3	최종 코드	12
8.4	실습	13
9	개념 연결	13
10	종합 정리	14
11	실습 파일 구성	15

1. 학습 목표

학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- `ofstream`을 이용해 파일에 데이터를 저장하기
- `ifstream`을 이용해 파일의 데이터를 읽기
- `ios::app`을 이용해 기존 파일 끝에 데이터를 추가하기
- `getline()`과 스트림 추출 연산자 `>>`의 차이 이해하기
- 파일의 데이터를 여러 변수로 나누어 읽는 File Parsing 수행하기
- `try`와 `catch`를 이용해 예외 처리 구조 작성하기
- `throw`와 `runtime_error`를 이용해 예외 발생시키기
- `exception`과 `what()`을 이용해 예외 메시지 처리하기

2. Day 13 개요

Day 13에서는 프로그램 외부의 파일과 데이터를 주고받는 File I/O와 실행 중 발생할 수 있는 문제를 처리하는 Exception Handling을 학습한다. 파일 출력에서는 `ofstream`, 파일 입력에서는 `ifstream`을 사용하고, 기존 파일의 내용을 유지하면서 데이터를 추가할 때는 `ios::app`을 사용한다. 또한 파일의 데이터를 변수로 분리하여 처리하는 File Parsing을 학습한다. 마지막으로 `try`, `catch`, `throw`를 이용한 기본적인 예외 처리 방법을 다룬다.

주제	핵심 역할
<code>ofstream</code>	파일에 데이터 쓰기
<code>ifstream</code>	파일에서 데이터 읽기
<code>ios::app</code>	기존 파일 끝에 데이터 추가
File Parsing	파일 데이터를 변수로 분리
<code>try</code> / <code>catch</code>	예외 발생 가능 코드와 처리 코드 분리
<code>throw</code>	예외 발생시키기

핵심 포인트

Day 13의 핵심은 파일 스트림을 이용해 외부 데이터를 읽고 쓰는 흐름과, 예외가 발생했을 때 프로그램의 제어 흐름이 `catch`로 이동하는 구조를 이해하는 것이다.

3. File Output: ofstream

`ofstream`은 output file stream의 의미로, 프로그램에서 파일에 데이터를 출력할 때 사용한다. 파일 입출력 기능을 사용하려면 `<fstream>` 헤더가 필요하다.

3.1. 파일 열기와 데이터 쓰기

다음 코드는 `data.txt` 파일을 연다.

```
1 ofstream file("data.txt");
```

파일이 존재하지 않으면 새 파일을 만들 수 있다. 파일에 데이터를 쓸 때는 `cout`과 마찬가지로 `<<` 연산자를 사용한다.

```
1 file << "Hello" << endl;
2 file << 100 << endl;
```

파일이 정상적으로 열렸는지는 `is_open()`으로 확인할 수 있다.

```
1 if (!file.is_open())
2 {
3     cout << "Failed to open file." << endl;
4     return 1;
5 }
```

`main()`에서 `return 0;`은 정상 종료를 의미하고, 0이 아닌 값은 일반적으로 오류가 발생한 종료 상태를 나타낸다. 파일 사용이 끝나면 `close()`로 닫는다.

3.2. 최종 코드

```
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4
5 using namespace std;
6
7 int main()
8 {
9     ofstream file("data.txt");
10
11     if (!file.is_open())
12     {
13         cout << "Failed to open file." << endl;
14         return 1;
15     }
16
17     string name = "Alice";
18     int score = 95;
19
20     file << "Name: " << name << endl;
21     file << "Score: " << score << endl;
22
23     file.close();
24
25     cout << "File saved successfully." << endl;
26
27     return 0;
28 }
```

파일에는 다음 내용이 저장된다.

Name: Alice

Score: 95

3.3. 실습

실습

`student.txt` 파일을 생성하고 Bob의 이름, 전공, 점수를 저장한다. 파일이 정상적으로 열렸는지 확인하고 저장이 끝나면 파일을 닫는다.

```

1 #include <iostream>
2 #include <fstream>
3 #include <string>
4
5 using namespace std;
6
7 int main()
8 {
9     ofstream file("student.txt");
10
11     if (!file.is_open())
12     {
13         cout << "Failed to open file." << endl;
14         return 1;
15     }
16
17     string name = "Bob";
18     string major = "Physics";
19     int score = 88;
20
21     file << "Name: " << name << endl;
22     file << "Major: " << major << endl;
23     file << "Score: " << score << endl;
24
25     file.close();
26
27     cout << "Student data saved " << endl;
28
29     return 0;
30 }

```

4. File Input: ifstream

ifstream은 input file stream의 의미로, 파일에 저장된 데이터를 프로그램으로 읽어올 때 사용한다. cin이 키보드 입력을 받는다면 ifstream은 파일에서 입력을 받는다.

4.1. 파일 열기

다음과 같이 읽을 파일을 지정한다.

```
1 ifstream file("data.txt");
```

ifstream은 읽을 파일이 존재해야 하므로 파일이 정상적으로 열렸는지 확인하는 것이 중요하다.

4.2. getline으로 한 줄 읽기

>> 연산자는 공백을 기준으로 값을 나누어 읽는다. 공백을 포함한 한 줄 전체를 읽으려면 getline()을 사용한다.

```
1 string line;
2 getline(file, line);
```

파일의 모든 줄을 읽을 때는 다음 패턴을 사용할 수 있다.

```
1 while (getline(file, line))
2 {
3     cout << line << endl;
4 }
```

getline()이 한 줄을 읽는 데 성공하는 동안 반복하고, 더 이상 읽을 데이터가 없으면 반복문을 종료한다.

4.3. 최종 코드

```
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4
5 using namespace std;
6
7 int main()
8 {
9     ifstream file("data.txt");
10
11     if (!file.is_open())
12     {
13         cout << "Failed to open file." << endl;
14         return 1;
15     }
16
17     string line;
18
19     while (getline(file, line))
20     {
21         cout << line << endl;
22     }
23
24     file.close();
25
26     return 0;
27 }
```

4.4. 실습

실습

student.txt를 ifstream으로 열고, getline()과 while을 사용하여 파일의 모든 줄을 출력한다.

```
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4
5 using namespace std;
6
7 int main()
8 {
9     ifstream file("student.txt");
10
11     if (!file.is_open())
12     {
13         cout << "Failed to open file." << endl;
14         return 1;
15     }
16
17     string line;
```

```

18     while (getline(file, line))
19     {
20         cout << line << endl;
21     }
22
23     file.close();
24
25     return 0;
26 }
27

```

5. Append Mode: ios::app

기본 ofstream으로 기존 파일을 열면 이전 내용을 덮어쓰게 된다. 기존 내용을 유지하면서 파일 끝에 새로운 데이터를 추가하려면 ios::app을 사용한다.

5.1. 기본 구조

```
1 ofstream file("student.txt", ios::app);
```

app은 append를 의미하며, 이후 출력되는 내용은 기존 파일의 끝에 추가된다.

형태	동작
ofstream file("data.txt")	기존 내용을 덮어쓰며 출력
ofstream file("data.txt", ios::app)	기존 내용 뒤에 추가

5.2. 최종 코드

```

1 #include <iostream>
2 #include <fstream>
3 #include <string>
4
5 using namespace std;
6
7 int main()
8 {
9     ofstream file("student.txt", ios::app);
10
11     if (!file.is_open())
12     {
13         cout << "Failed to open file." << endl;
14         return 1;
15     }
16
17     string name = "Alice";
18     string major = "Mathematics";
19     int score = 95;
20
21     file << endl;
22     file << "Name: " << name << endl;
23     file << "Major: " << major << endl;
24     file << "Score: " << score << endl;
25
26     file.close();
27

```

```

28     cout << "Student data appended." << endl;
29
30     return 0;
31 }

```

5.3. 실습

실습

기존 `student.txt`에 Charlie의 정보를 추가한다. `ios::app`을 사용하고 기존 학생과 구분되도록 빈 줄 하나를 먼저 출력한다.

```

1  #include <iostream>
2  #include <fstream>
3  #include <string>
4
5  using namespace std;
6
7  int main()
8  {
9      ofstream file("student.txt", ios::app);
10
11     if (!file.is_open())
12     {
13         cout << "Failed to open file." << endl;
14         return 1;
15     }
16
17     file << endl;
18
19     string name = "Charlie";
20     string major = "ComputerScience";
21     int score = 91;
22
23     file << "Name: " << name << endl;
24     file << "Major: " << major << endl;
25     file << "Score: " << score << endl;
26
27     file.close();
28
29     cout << "Student data appended." << endl;
30
31     return 0;
32 }

```

6. File Parsing

File Parsing은 파일에 저장된 데이터를 단순한 한 줄 문자열로 읽는 것이 아니라, 각 데이터를 변수에 나누어 저장하여 프로그램에서 사용할 수 있도록 처리하는 것이다.

6.1. 스트림 추출 연산자로 데이터 분리

파일의 내용이 다음과 같다고 하자.

Alice 95

Bob 88
Charlie 91

다음 코드로 이름과 점수를 각각의 변수에 읽을 수 있다.

```
1 string name;
2 int score;
3
4 file >> name >> score;
```

여러 줄을 계속 읽으려면 입력 작업을 자체를 while의 조건으로 사용할 수 있다.

```
1 while (file >> name >> score)
2 {
3     cout << name << " " << score << endl;
4 }
```

6.2. 읽은 데이터 계산하기

읽은 점수를 합계와 개수에 반영할 수 있다.

```
1 int total = 0;
2 int count = 0;
3
4 while (file >> name >> score)
5 {
6     total += score;
7     count++;
8 }
```

평균을 계산할 때 정수 나눗셈을 피하려면 static_cast<double>을 사용한다.

```
1 double average = static_cast<double>(total) / count;
```

6.3. 최종 코드

```
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4
5 using namespace std;
6
7 int main()
8 {
9     ifstream file("scores.txt");
10
11     if (!file.is_open())
12     {
13         cout << "Failed to open file." << endl;
14         return 1;
15     }
16
17     string name;
18     int score;
19
20     int total = 0;
21     int count = 0;
22
23     while (file >> name >> score)
```

```

24 {
25     cout << "Name: " << name
26         << ", Score: " << score << endl;
27
28     total += score;
29     count++;
30 }
31
32 file.close();
33
34 if (count > 0)
35 {
36     double average = static_cast<double>(total) / count;
37
38     cout << "Total: " << total << endl;
39     cout << "Average: " << average << endl;
40 }
41
42 return 0;
43 }

```

6.4. 실습

실습

grades.txt에 저장된 이름과 점수를 읽어 각 학생의 점수를 출력하고, 전체 점수의 총합과 평균을 계산한다.

```

Tom 80
Jane 92
Mike 75
Emma 98

```

```

1 #include <iostream>
2 #include <fstream>
3 #include <string>
4
5 using namespace std;
6
7 int main()
8 {
9     ifstream file("grades.txt");
10
11     if (!file.is_open())
12     {
13         cout << "Failed to open file" << endl;
14         return 1;
15     }
16
17     string name;
18     int score;
19
20     int total = 0;
21     int count = 0;
22
23     while (file >> name >> score)
24     {

```

```

25     cout << name << ": " << score << endl;
26     total += score;
27     count++;
28 }
29
30 file.close();
31
32 if (count > 0)
33 {
34     double average = static_cast<double>(total) / count;
35
36     cout << "Total: " << total << endl;
37     cout << "Average: " << average << endl;
38 }
39
40 return 0;
41 }

```

7. Exception Handling: try와 catch

예외 처리는 프로그램 실행 중 발생할 수 있는 문제를 처리하는 방법이다. try 블록에는 예외가 발생할 수 있는 코드를 작성하고, 예외가 발생하면 대응되는 catch 블록에서 처리한다.

7.1. 기본 구조

```

1 try
2 {
3     throw 1;
4 }
5 catch (...)
6 {
7     cout << "Error occurred." << endl;
8 }

```

catch (...)는 예외의 종류와 관계없이 예외를 잡는다. throw가 실행되면 현재 try 블록의 나머지 코드는 실행되지 않고, 처리할 수 있는 catch 블록으로 제어가 이동한다.

7.2. 최종 코드

```

1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     try
8     {
9         cout << "Try block started." << endl;
10
11         throw 1;
12
13         cout << "Try block finished." << endl;
14     }
15     catch (...)
16     {
17         cout << "Exception caught." << endl;
18     }

```

```

19     cout << "Program continues." << endl;
20
21     return 0;
22 }
23

```

출력 결과는 다음과 같다.

```

Try block started.
Exception caught.
Program continues.

```

7.3. 실습

실습

try 블록에서 Start를 출력한 뒤 throw 100;으로 예외를 발생시킨다. catch (...)에서 예외를 처리하고, 이후 프로그램이 계속 실행되는지 확인한다.

```

1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     try
8     {
9         cout << "Start" << endl;
10        throw 100;
11        cout << "End" << endl;
12    }
13    catch (...)
14    {
15        cout << "Exception detected." << endl;
16    }
17
18    cout << "Program finished." << endl;
19
20    return 0;
21 }

```

8. throw와 Standard Exception

단순한 정수도 throw할 수 있지만, 실제 오류 정보를 전달하려면 표준 예외 클래스를 사용할 수 있다. Day 13에서는 runtime_error와 exception을 사용한다.

8.1. runtime_error

runtime_error를 사용하려면 <stdexcept> 헤더가 필요하다.

```

1 throw runtime_error("Invalid value.");

```

예외는 다음과 같이 받을 수 있다.

```

1 catch (const exception& e)
2 {
3     cout << e.what() << endl;
4 }

```

`const exception&`는 예외 객체를 복사하지 않고 참조로 받으며, `what()`은 예외 객체에 저장된 오류 메시지를 반환한다. `runtime_error`는 표준 `exception` 계열의 예외이므로 `catch (const exception& e)`로 처리할 수 있다.

8.2. 함수에서 예외 발생시키기

예외를 발견하는 코드와 처리하는 코드는 서로 다른 함수에 있을 수 있다.

```
1 double divide(double a, double b)
2 {
3     if (b == 0)
4     {
5         throw runtime_error("Cannot divide by zero.");
6     }
7
8     return a / b;
9 }
```

함수에서 발생한 예외는 호출한 쪽의 `catch`에서 처리할 수 있다.

8.3. 최종 코드

```
1 #include <iostream>
2 #include <stdexcept>
3
4 using namespace std;
5
6 double divide(double a, double b)
7 {
8     if (b == 0)
9     {
10         throw runtime_error("Cannot divide by zero.");
11     }
12
13     return a / b;
14 }
15
16 int main()
17 {
18     double a;
19     double b;
20
21     cout << "Enter two numbers: ";
22     cin >> a >> b;
23
24     try
25     {
26         double result = divide(a, b);
27
28         cout << "Result: " << result << endl;
29     }
30     catch (const exception& e)
31     {
32         cout << "Error: " << e.what() << endl;
33     }
34
35     return 0;
36 }
```

8.4. 실습

실습

checkScore(int score) 함수를 만들고 점수가 0보다 작거나 100보다 크면 runtime_error를 발생시킨다. 정상 범위의 점수라면 점수를 출력하고, 예외는 main()의 catch에서 처리한다.

```

1 #include <iostream>
2 #include <stdexcept>
3
4 using namespace std;
5
6 void checkScore(int score)
7 {
8     if (score < 0 || score > 100)
9     {
10         throw runtime_error("Score must be between 0 and 100.");
11     }
12
13     cout << "Valid score: " << score << endl;
14 }
15
16 int main()
17 {
18     int score;
19     cout << "Enter a score: ";
20     cin >> score;
21
22     try
23     {
24         checkScore(score);
25     }
26     catch (const exception& e)
27     {
28         cout << "Error: " << e.what() << endl;
29     }
30
31     return 0;
32 }

```

9. 개념 연결

Day 13의 각 기능은 이전에 배운 C++ 문법과 연결된다.

이전 학습 내용	Day 13에서의 연결
스트림 입출력	cin, cout에서 파일 스트림으로 확장
참조	catch (const exception& e)
클래스와 상속	runtime_error와 exception
함수	함수 내부에서 throw하고 호출한 쪽에서 처리
형 변환	평균 계산에서 static_cast<double> 사용

핵심 포인트

`ofstream`과 `ifstream`은 기존의 스트림 입출력 문법을 파일로 확장한 것이다. 예외 처리에서는 문제가 발생한 위치에서 `throw`하고, 처리할 위치에서 `catch`하여 프로그램의 오류 처리 흐름을 분리할 수 있다.

10. 종합 정리

핵심 포인트

- `ofstream`은 파일에 데이터를 출력할 때 사용한다.
- `ifstream`은 파일의 데이터를 읽을 때 사용한다.
- 파일이 정상적으로 열렸는지는 `is_open()`으로 확인할 수 있다.
- `getline()`은 공백을 포함하여 한 줄 전체를 읽는다.
- `ios::app`은 기존 파일 내용을 유지하면서 파일 끝에 데이터를 추가한다.
- `file >> variable` 형태를 이용하면 파일 데이터를 변수로 나누어 읽을 수 있다.
- 평균 계산에서 정수 나눗셈을 피하려면 `static_cast<double>`을 사용할 수 있다.
- `try`에는 예외가 발생할 수 있는 코드를 작성하고 `catch`에서 예외를 처리한다.
- `throw`가 실행되면 현재 `try`의 나머지 코드는 실행되지 않는다.
- `runtime_error`는 오류 메시지를 포함한 예외를 발생시키는 데 사용할 수 있다.
- `catch (const exception& e)`로 표준 예외를 받고 `e.what()`으로 메시지를 확인할 수 있다.

11. 실습 파일 구성

실습

Day 13 파일은 다음과 같이 관리한다.

- 01_file_output.cpp
- 02_file_input.cpp
- 03_file_append.cpp
- 04_file_parsing.cpp
- 05_exception_basic.cpp
- 06_throw_catch.cpp
- data.txt
- scores.txt
- practice/p01_file_output.cpp
- practice/p02_file_input.cpp
- practice/p03_file_append.cpp
- practice/p04_file_parsing.cpp
- practice/p05_exception_basic.cpp
- practice/p06_throw_catch.cpp
- practice/student.txt
- practice/grades.txt